TECHNICAL RESEARCH DOSSIER & IMPLEMENTATION PLAN

Integrating Unipile into Project Beacon

A fact-checked, source-scored deep dive into Unipile's authentication pipeline, account lifecycle, webhooks, provider limits and platform risk — plus a concrete, phased build plan for the G3 recruitment platform.

PREPARED FOR	DATE
Project Beacon / G3 Recruitment Automation	31 July 2026
TARGET STACK	**DOCUMENT TYPE**
TanStack Start (React 19) · Nitro/Cloudflare · Prisma · PostgreSQL	Research + Proposal (no code shipped)
PRIMARY SOURCES	**SECONDARY SOURCES**
18 official Unipile docs / OpenAPI pages fetched directly	14 third-party reviews, comparisons, GitHub issues
SUPERSEDES	**CONFIDENCE SCALE**
Unipile_Authentication_and_Subscription_Management_Implementation_Plan.pdf	Every claim carries a source-confidence badge (see §0)

The one-paragraph version. Unipile is a French (Scaleway-hosted, SOC 2 Type II, GDPR-processor) unified messaging API that lets Project Beacon read and write a recruiter's *own* LinkedIn, Gmail/Outlook and WhatsApp inbox without ever touching their password. The integration itself is small — one `POST` to mint a hosted-auth link, one webhook endpoint, one new database table. The hard parts are not the API: they are **(a)** two *different* webhook payload shapes that the existing plan conflates, **(b)** the absence of any documented HMAC signature on webhooks, **(c)** LinkedIn's per-account action ceilings (~80–100 invites/day, ~100 profile fetches/day) which are the real throughput limit on the whole product, and **(d)** peak-based billing that makes seat control a financial control, not a UX nicety. This document verifies each of those against primary sources and gives a five-phase build plan keyed to real files in this repository.

CONTENTS

Method

Every factual claim below was taken from a source I fetched directly during this research pass — not from prior knowledge. Where the official documentation and Unipile's own marketing site disagree, both are quoted and the disagreement is flagged rather than resolved silently. Where only third-party SEO-driven blogs make a claim, the claim is marked as such and *not* used as a load-bearing input to the plan.

Confidence scale

Each source in Appendix A and each significant claim in the body carries one of four badges:

BADGE	SCORE	MEANING AND HOW IT IS ALLOWED TO BE USED
HIGH	90–99	Unipile's own developer documentation or OpenAPI reference (`developer.unipile.com`), including the machine-readable `llms.txt` index and `.md` doc endpoints. **Safe to build on.** Residual risk is only that docs drift from runtime behaviour.
MEDIUM	60–89	Unipile's commercial site (pricing, security pages), the official GitHub SDK and its issue tracker, or a named third-party review with checkable specifics. **Usable, verify before it becomes a contract term.**
LOW	25–59	Competitor or affiliate content, comparison blogs with commercial incentive, unattributed statistics. **Directional signal only — never a build input.**
UNVERIFIED	<25	Could not be fetched (paywall, 403, robots), or the claim was found only in aggregate search snippets. **Recorded for completeness, explicitly excluded from the plan.**

TWO HONEST LIMITATIONS OF THIS RESEARCH

1. Reddit and Stack Overflow are effectively empty on Unipile. Direct fetches of Reddit were blocked in this environment, and repeated site-scoped searches (`site:reddit.com unipile` , plus subreddit-targeted queries) returned *zero* genuine Reddit threads — search engines instead substituted alternative-listing sites. Likewise no Stack Overflow question about Unipile surfaced. §10 treats this absence as a finding in its own right rather than papering over it with invented quotes.

2. G2 returned HTTP 403. The review aggregate could not be read first-hand; the two ratings cited (4.3 and 4.6 of 5) come from third parties that disagree with each other, and are marked LOW accordingly.

What Unipile is

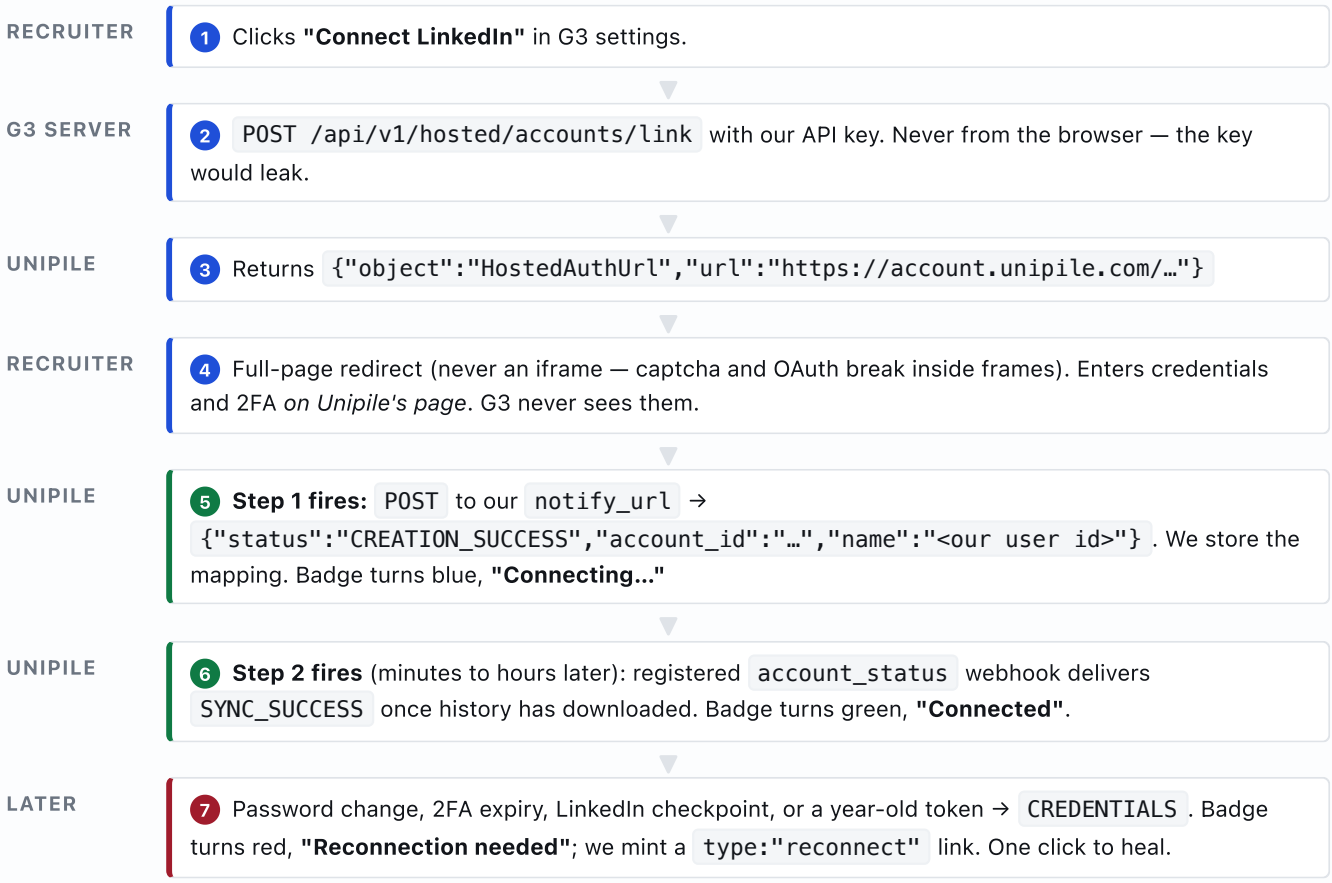
Unipile is a single API that sits in front of channels that have no usable official API for this purpose — most importantly **LinkedIn**, plus Gmail, Outlook, IMAP, WhatsApp, Instagram, Telegram, Messenger, X, and Google/Outlook calendars. A recruiter connects their *own* account once; from then on Project Beacon can read that inbox and send from it over plain HTTPS. Unipile is not a scraper-for-hire and not a lead database: it is deliberately messaging-shaped. HIGH

Why it fits Project Beacon specifically

The V2 Prisma schema in `server/Project_Beacon_DB_schema_V2.prisma` already anticipates it. `InteractionChannel` contains `LINKEDIN_DM`; the comment on `InteractionEvent.deliveryStatus` literally reads *"(Resend webhooks / Unipile webhooks)"*; `ConversationChannel` already lists LinkedIn, Instagram and WhatsApp. The integration is therefore not a new capability bolted on — it is the missing engine under UI and data models that were designed for it.

How a recruiter connects — the two-step flow

FIGURE 1 — HOSTED-AUTH CONNECTION, EXACTLY AS DOCUMENTED



THE SINGLE MOST IMPORTANT CORRECTION IN THIS DOCUMENT

Steps 5 and 6 arrive on **two different channels with two incompatible JSON shapes**. The hosted-auth `notify_url` callback is flat — `{"status": "...", "account_id": "...", "name": "..."}`. The ongoing lifecycle webhook is nested and uses a different key name for the status — `{"AccountStatus": {"account_id": "...", "account_type": "...", "message": "..."}}`. A single handler that reads only `body.status` will silently ignore every lifecycle event, including `CREDENTIALS` — meaning broken accounts would never show as broken. The existing plan's `normalizeUnipileStatus(rawMessage)` helper is correct in spirit but is only wired to one of the two shapes. HIGH

What it costs, and why that is an engineering concern

€49/month covers up to 10 linked accounts; beyond that roughly €5 per account per month. Critically, invoices are cut at the end of each 30-day period and are based on the **peak number of accounts linked simultaneously** during that window — not the count on the last day. One recruiter who links LinkedIn + Gmail + WhatsApp is **three** billed accounts. Gmail and Outlook bundle mail + calendar as one. Deleting a seat mid-month therefore does not reduce that month's bill; it only lowers future peaks. Seat capping has to be enforced server-side at link-generation time. MEDIUM

What will actually limit the product

Not Unipile — it states it imposes no usage limits of its own. LinkedIn does. Roughly **80–100 connection invites per day and ~200 per week** per paid account; **~100 profile fetches per day**; a single search returns at most **1,000 profiles** (2,500 with Sales Navigator or Recruiter). Unipile's own guide works the arithmetic: a 1,000-profile outreach sequence takes **about five weeks**. Any recruiter-facing promise about outreach volume must be built on those numbers, and the queue must be designed to spread and randomise sends rather than drain as fast as it can. HIGH

The honest risk summary

- **LinkedIn's terms.** This is unofficial access to LinkedIn, delegated by the account holder. It is a category LinkedIn's user agreement disfavours, and 2026 saw enforcement move from individual accounts to *vendors*. The mitigation is behavioural — human-rate volumes, no pure-write bursts — not contractual.
- **No documented webhook signature.** Unipile's OpenAPI spec for `POST /webhooks` exposes no secret, no HMAC field. Its marketing site claims "Every webhook is signed." Until support confirms, the only documented defence is a custom shared-secret header plus a hard-to-guess URL. Design for that. (§6)
- **Thin public community.** No Reddit, no Stack Overflow. Support is the vendor, so their responsiveness is a real dependency — evaluate it during the 7-day trial.

2 · AUDIT OF THE EXISTING IMPLEMENTATION PLAN

Target: `Unipile_Authentication_and_Subscription_Management_Implementation_Plan.md/.pdf` (dated 31 July 2026, in the repo root). Verdict: **directionally correct and unusually well-shaped for a first pass** — the two-step CONNECTING→OK model, the normalisation layer, and the peak-billing insight are all right. Nine items need correcting before it can be built from.

2.1 What the existing plan gets right — keep all of this

CLAIM IN THE PLAN	VERIFICATION	CONF.
G3 never sees passwords, 2FA codes or session tokens	Correct for hosted auth. Credentials are entered on Unipile's hosted page; our server only ever receives an `account_id`.	HIGH
Two-step setup: `CREATION_SUCCESS` → then `OK` / `SYNC_SUCCESS`	Correct and well-observed. Docs: `CREATION_SUCCESS` = "account was successfully added and the initial data synchronization has begun"; `SYNC_SUCCESS` = resynchronisation finished.	HIGH
`CREDENTIALS` means re-auth needed; prompt the user	Correct. Docs list triggers: password change, revoked authorisation, token expiry (LinkedIn/Instagram after one year), session termination, excessive API requests, proxy issues, security checkpoints.	HIGH
Normalise vendor strings to a small internal enum	Right instinct, and more necessary than the plan realises — there are 11 distinct events, not 8.	HIGH
Store `rawUnipileStatus` as an audit trail	Excellent. Given the 11-event surface and undocumented drift risk, keeping the raw string is the difference between a debuggable and an undebuggable integration.	HIGH
Billing is peak simultaneous accounts over a rolling 30-day window	**Confirmed verbatim:** "invoices are generated at the end of each 30-day period. Billing is based on the peak number of linked accounts active simultaneously during that period."	MEDIUM
Deleting an account frees a seat for *future* peaks only	Correct inference from the above. This is a subtle point the plan gets right and most integrations get wrong.	MEDIUM

2.2 Corrections required

#	ISSUE	CORRECTION & WHY IT MATTERS
C1 BLOCKER	One handler assumed for both callbacks	Two shapes, as set out in §1. Hosted-auth `notify_url`: flat, key `status`. Lifecycle webhook: nested under `AccountStatus`, key `message`. Read `body.AccountStatus?.message ?? body.status`, never just one.
C2 BLOCKER	Registering a webhook is not mentioned at all	`notify_url` only ever fires for creation/reconnection of *that one link*. Ongoing events (`CREDENTIALS`, `SYNC_SUCCESS`, `ERROR`, `PERMISSIONS`) require a separately registered webhook: `POST /api/v1/webhooks` with `source:"account_status"`. Without it, the self-healing reconnection flow in the plan can never trigger.

#	ISSUE	CORRECTION & WHY IT MATTERS
C3 MAJOR	Status list is incomplete — 2 events missing	The `account_status` source emits eleven events: `creation_success, creation_fail, deleted, reconnected, sync_success, stopped, ok, connecting, error, credentials, permissions`. The plan's normaliser handles neither **`CREATION_FAIL`** (recruiter's connection attempt failed — needs its own UI state, currently would fall through to the amber "Sync Stopped" catch-all and read as a system fault rather than a failed login) nor **`PERMISSIONS`** (OAuth scope was declined/revoked — for Google/Microsoft this needs a re-consent prompt, not a password prompt).
C4 MAJOR	`'ERROR / STOPPED'` treated as a literal string	That is documentation prose describing two events, not a wire value. `error` and `stopped` arrive separately. Matching on the literal `'ERROR / STOPPED'` is dead code; matching on `ERROR` and `STOPPED` individually is what is needed. (The plan does also list those, so the effect is cosmetic — but it signals the list was assembled from prose rather than the event enum.)
C5 BLOCKER	`model Organization` does not exist in the real schema	The plan hangs `maxUnipileSlots` off an `Organization` model and gives `ConnectedAccount` an `orgId`. `Project_Beacon_DB_schema_V2.prisma` has **no Organization model and no tenancy concept** — `User` has no `orgId`. Introducing one is a migration touching `User` and every ownership relation, for a single integer. The schema already ships `model SystemConfig { key, value, notes }` for exactly this: store `unipile.max_slots` there. Defer real multi-tenancy to when a second tenant exists. (§13)
C6 MAJOR	Provider enum does not match Unipile's	Plan has `LINKEDIN, GOOGLE, MICROSOFT, WHATSAPP`. Unipile's hosted-auth `providers` enum is `LINKEDIN, WHATSAPP, INSTAGRAM, MESSENGER, TELEGRAM, GOOGLE, OUTLOOK, MAIL, TWITTER` — note **`OUTLOOK`**, **not `MICROSOFT`**, and `MAIL` for generic IMAP. Mismatched enums mean the webhook's `account_type` will not parse into our type.
C7 MAJOR	No idempotency; `SYNC_SUCCESS` can arrive twice	Documented: LinkedIn **Premium accounts receive two `SYNC_SUCCESS` payloads**, distinguished by a `product` field. Add that webhooks retry **five times** whenever we fail to return 200 within 30 seconds, and duplicate delivery is routine, not exceptional. Needs a dedupe/event-log table and idempotent writes.
C8 MAJOR	Nothing about link lifetime or single use	Two documented behaviours the plan needs: `expiresOn` is mandatory and must match `^[1-2]\d{3}-[0-1]\d-[0-3]\dT\d{2}:\d{2}:\d{2}\.\d{3}Z$`; and *all links expire on Unipile's daily restart regardless of the stated expiry*. So a link minted at 23:50 may die minutes later — mint on demand with a short TTL (15 min) and set `single_use: true`. Never email a link or cache one.

#	ISSUE	CORRECTION & WHY IT MATTERS
C9 MAJOR	Seat cap counts the wrong unit; and no `name` mapping	Two things. (i) The plan counts "active accounts" per organisation but bills per *linked account*; one recruiter connecting three providers consumes three seats. Cap on rows in `ConnectedAccount`, and cap per user too, or one recruiter can exhaust the org. (ii) The plan never mentions the `name` parameter — it is the *only* mechanism that ties a hosted-auth callback back to our user. Pass an opaque, single-use nonce (not the raw user id — it is echoed back through a URL under the recruiter's control).

2.3 Corrected normaliser

```
// server/lib/unipile/status.ts
export type AccountStatus =
  | 'CONNECTING' | 'OK' | 'RECONNECTION_NEEDED'
  | 'PERMISSION_REVOKED'   // C3 — new: OAuth scope declined/revoked
  | 'CONNECT_FAILED'       // C3 — new: the recruiter's attempt itself failed
  | 'SYNC_STOPPED' | 'DISCONNECTED';

/** Accepts BOTH callback shapes — see C1. */
export function extractUnipileStatus(body: unknown): string | undefined {
  const b = body as Record<string, any> | null;
  return b?.AccountStatus?.message  // lifecycle webhook  (nested, key = message)
      ?? b?.status;                 // hosted-auth notify_url (flat, key = status)
}

export function normalizeUnipileStatus(raw?: string): AccountStatus {
  if (!raw) return 'DISCONNECTED';
  const s = raw.toUpperCase().trim().replace(/[\s/]+/g, '_');

  switch (s) {
    case 'CREATION_SUCCESS':
    case 'CONNECTING':          return 'CONNECTING';
    case 'OK':
    case 'SYNC_SUCCESS':
    case 'RECONNECTED':         return 'OK';
    case 'CREDENTIALS':         return 'RECONNECTION_NEEDED';
    case 'PERMISSIONS':         return 'PERMISSION_REVOKED';
    case 'CREATION_FAIL':       return 'CONNECT_FAILED';
    case 'ERROR':
    case 'STOPPED':             return 'SYNC_STOPPED';
    case 'DELETED':
    case 'DISCONNECTED':        return 'DISCONNECTED';
    default:
      // Unknown vendor string: park in SYNC_STOPPED but ALERT — do not fail silently.
      console.error('[unipile] unmapped account status', { raw });
      return 'SYNC_STOPPED';
  }
}
```

2.4 Corrected badge matrix

Connecting…
Blue, spinner. From `CREATION_SUCCESS` / `CONNECTING`.
No user action. Show "history is downloading — this can take a while."

Connected
Green. From `OK` / `SYNC_SUCCESS` / `RECONNECTED`.
Action: Disconnect.

Reconnection needed
Red. From `CREDENTIALS`. Action: **Reconnect** (mints `type:"reconnect"` link). Also email the recruiter.

Permission revoked
New (C3). Red. From `PERMISSIONS`. Action: **Re-grant access** — an OAuth consent problem, not a password problem. Different copy.

Couldn't connect
New (C3). Amber. From `CREATION_FAIL`. Action: **Try again**. Distinguishes "your login failed" from "our sync broke".

Sync stopped
Amber. From `ERROR` / `STOPPED` and any unmapped string. Action: Reconnect / contact support.

Not connected
Grey. From `DELETED` or no row. Action: **Connect account**.

ONE BEHAVIOURAL NOTE THE UI MUST CARRY

Docs, on any non-`OK` status: reads via `GET` keep working but "data retrieval is no longer guaranteed current, and webhook message updates cease." So a red or amber account is not merely cosmetic — **inbound replies silently stop arriving**. Any SLA metric in the scoring rubric (`sla_adherence` , `time_to_first_touch`) computed over a window where the account was degraded is invalid and should be excluded, exactly the way `RecruiterScoreSnapshot.excludedMetrics` already allows for.

3 · PLATFORM FUNDAMENTALS

All items in this section: HIGH — from `developer.unipile.com` docs and OpenAPI reference.

Sign-up	`dashboard.unipile.com/signup` — 7-day free trial, no card.
Access token	Created at `dashboard.unipile.com/access-tokens`.
DSN	Your tenant's Data Source Name from the dashboard. Every request is routed through it. Shaped `apiXXX.unipile.com:XXXX` — note the **non-standard port**.
Base URL	`https://{YOUR_DSN}/api/v1`
Port workaround	If egress firewalls block the custom port: `https://apiX.unipile.com/api/v1/accounts?port=XXXXX` over standard 443. *Relevant to Cloudflare Workers, which restrict outbound ports — see §12.*
Auth header	`X-API-KEY: <access token>`. Not `Authorization: Bearer`.
Other headers	`accept: application/json`; `content-type: application/json` (messaging send endpoints use `multipart/form-data`).
Pagination	Opaque `cursor` in the response; pass it back for the next page. `cursor === null` means no more results.
Node SDK	`npm i unipile-node-sdk` (Node 18+). `new UnipileClient('https://{DSN}', '{TOKEN}')`. Namespaces: `account`, `messaging`, `users`, `email`. Escape hatches: `extra_params` on any method, and `client.request.send()` for endpoints not yet wrapped.
MCP server	`https://developer.unipile.com/mcp?branch=v1.0`, authenticated with `X-API-KEY`. Gives Cursor/Claude live API access, doc search and code generation. Useful during build; not a runtime dependency.

Error taxonomy (from the OpenAPI spec for `POST /accounts`)

HTTP	TYPE	HANDLING
400	`errors/invalid_parameters` · `missing_parameters` · `invalid_request` · `malformed_request` · `content_too_large` · `too_many_characters` · `unescaped_characters` · `limit_too_high`	Our bug. Log loudly, do not retry.
401	`errors/invalid_credentials`	Either our API key is wrong, or the recruiter's provider credentials are. Disambiguate by endpoint.
407	`errors/proxy_auth_error`	Proxy config. Only if we supply our own proxies (we should not initially).
408	`errors/request_timeout`	Retry with backoff.
425	`errors/auth_in_progress`	**Too Early.** An auth attempt for this account is already running. Do not mint a second link — surface "already connecting".
422	`cannot_resend_yet`	LinkedIn invitation limit hit. Requeue, do not retry immediately. (§8)
429 / 500	`errors/unexpected_error` · `provider_error` · `authentication_intent_error`	Provider-side throttling or fault. Exponential backoff + jitter; escalate after N.

The SDK surfaces these as `UnsuccessfulRequestError`. **Known limitation:** open issue #16 — *"UnsuccessfulRequestError type does not expose body.{status,type}"*. So the machine-readable error type above is present on the wire but awkward to read through the SDK's types. Practical consequence for us: **call the REST API directly with `fetch`** rather than adopting the SDK (§12 argues this on other grounds too).

4.1 Two connection methods — and which to choose

	HOSTED AUTH WIZARD RECOMMENDED	CUSTOM / NATIVE AUTH
Endpoint	`POST /api/v1/hosted/accounts/link`	`POST /api/v1/accounts`
Who sees credentials	Unipile only.	**Our servers.** We would be transiting LinkedIn passwords and li_at cookies.
Handles	QR codes, credentials, OAuth, 2FA, OTP, captcha, checkpoints — all of it, maintained by Unipile.	We must build every checkpoint UI ourselves: `2FA, OTP, IN_APP_VALIDATION, CAPTCHA, PHONE_REGISTER`, plus `TRY_ANOTHER_WAY` fallback, plus a 5-minute intent expiry timer.
Effort	One POST + one webhook.	Weeks, and it never stops changing as LinkedIn changes.
Compliance	Keeps provider credentials entirely out of Project Beacon's scope.	Puts them squarely in scope — a materially worse posture for a platform holding candidate PII.
Verdict	**Use hosted auth. There is no serious argument for native auth here.** The only scenario that would force it is a white-glove flow where a recruiter's LinkedIn Recruiter seat conflicts with concurrent sessions — for which the documented remedy is cookie-based auth. Treat that as a support escalation, not a v1 feature.	

4.2 Complete request schema — `POST /api/v1/hosted/accounts/link`

Two mutually exclusive variants (`anyOf`). Required fields in **bold**.

FIELD	TYPE	NOTES
`type`	`"create"` | `"reconnect"`	Selects the variant.
`api_url`	string	`https://{subdomain}.unipile.com:{port}` — our DSN. Tells the hosted page which tenant to attach to.
`expiresOn`	ISO-8601 UTC	Strict pattern `^[1-2]\d{3}-[0-1]\d-[0-3]\dT\d{2}:\d{2}:\d{2}\.\d{3}Z$` — milliseconds and trailing `Z` are mandatory. `new Date().toISOString()` matches. **Links also die on Unipile's daily restart regardless of this value.**
`providers` `(create only)`	string | array	`"*"` · `"*:MAILING"` · `"*:MESSAGING"` · `"*:CALENDAR"` · or an array from `LINKEDIN, WHATSAPP, INSTAGRAM, MESSENGER, TELEGRAM, GOOGLE, OUTLOOK, MAIL, TWITTER`. *Always pass an explicit single-element array* so the recruiter cannot connect a channel we do not bill or support.
`reconnect_account` `(reconnect only)`	string	The `account_id` to heal.
`name`	string	Echoed back on `notify_url`. **The only correlation key.** Pass an opaque single-use nonce.
`notify_url`	string	Server-to-server callback. Must be publicly reachable.

FIELD	TYPE	NOTES
`success_redirect_url` `failure_redirect_url`	string	Where the recruiter's browser lands. **Never treat arrival here as proof of success** — it is user-navigable. Truth comes from `notify_url`.
`bypass_success_screen`	boolean	Skip Unipile's interstitial and redirect straight back. Nicer UX; set `true`.
`single_use`	boolean	Link works once. Set `true`.
`disabled_features`	array	`linkedin_recruiter`, `linkedin_sales_navigator`, `linkedin_organizations_mailboxes`. Disable what we don't use — less surface, fewer sessions, fewer Recruiter-seat conflicts.
`disabled_options`	array	`proxy, autoproxy, cookie_auth, credentials_auth, qrcode_auth, pairing_code_auth, sync_limit, language`. **Disable `cookie_auth`** — otherwise the wizard invites recruiters to paste their raw `li_at` cookie, which is both a worse security story and (per §11) a higher-detection-risk pattern.
`sync_limit`	object	`{ MAILING: "NO_HISTORY_SYNC", MESSAGING: { chats: <ISO|int>, messages: <ISO|int> } }`. Accepts a date or a count; `0` syncs no history. **Materially reduces first-connect time and our storage.** Recommend ~90 days of chats for recruiting context.
`proxy`	object	`{ protocol: https|http|socks5, host, port, username?, password? }`. Leave unset in v1; let Unipile manage egress IPs.
`google_scopes` `microsoft_scopes`	string	Comma-separated OAuth scopes. Request the minimum — over-broad scopes are what later produce `PERMISSIONS` revocations.

RESPONSE — 200

```
{ "object": "HostedAuthUrl", "url": "https://account.unipile.com/<encoded>" }
```

Note: the docs page renders `"HostedAuthURL"` in prose and `"HostedAuthUrl"` in the OpenAPI schema. Match on `url` being present, not on the `object` string.

WORKED EXAMPLE — WHAT WE WILL ACTUALLY SEND

```
POST https://api8.unipile.com:13081/api/v1/hosted/accounts/link
X-API-KEY: <UNIPILE_API_KEY>
content-type: application/json

{
  "type": "create",
  "providers": ["LINKEDIN"],
  "api_url": "https://api8.unipile.com:13081",
  "expiresOn": "2026-07-31T11:15:00.000Z",        // now + 15 min
  "name": "b7c1e2f4-…",                            // opaque nonce -> our user
  "notify_url": "https://app.g3.example/api/unipile/webhook",
  "success_redirect_url": "https://app.g3.example/recruiter/settings?connected=1",
  "failure_redirect_url": "https://app.g3.example/recruiter/settings?connected=0",
  "bypass_success_screen": true,
  "single_use": true,
  "disabled_options": ["cookie_auth", "proxy", "autoproxy"],
  "disabled_features": ["linkedin_organizations_mailboxes"],
  "sync_limit": { "MESSAGING": { "chats": 90, "messages": 500 } }
}
```

THREE DOCUMENTED IMPLEMENTATION RULES — ALL EASY TO GET WRONG

1. **Generate links server-side only.** The call carries our API key; minting from the browser leaks it to every recruiter.
2. **Do not embed the wizard in an iframe.** Docs are explicit: captcha and OAuth break. Full-page redirect or a new tab.
3. **Watch for `CREDENTIALS` continuously** — the reconnect flow is not optional polish; an unattended `CREDENTIALS` means an account that looks fine and silently sends nothing.

4.3 For completeness — native auth surface

Not recommended (§4.1), documented here so the decision is informed. `POST /api/v1/accounts`, LinkedIn has two variants:

```
// (a) credentials
{ "provider":"LINKEDIN", "username":"<email|phone>", "password":"…",
  "country":"FR", "ip":"1.2.3.4", "user_agent":"…", "recruiter_contract_id":"…",
  "proxy":{ "protocol":"https","host":"…","port":8080,"username":"…","password":"…" },
  "disabled_features":
["linkedin_recruiter","linkedin_sales_navigator","linkedin_organizations_mailboxes"],
  "sync_limit":{ "chats":90, "messages":500 } }

// (b) session cookie
{ "provider":"LINKEDIN", "access_token":"<li_at>", "premium_token":"<li_a>",
  "user_agent":"…", /* + same optional fields */ }

// success
{ "object":"AccountCreated", "account_id":"…" }
```

Other providers, for reference: `GOOGLE_OAUTH` and `OUTLOOK` take `refresh_token` + `access_token`; `MAIL` takes the full IMAP/SMTP sextet (`imap_user/host/port/password` , `smtp_*` , `imap_encryption`); `WHATSAPP` optionally takes `pairing_phone_number` in E.164; `MESSENGER` accepts a raw cookie jar. Checkpoint intents expire after **5 minutes**, and automatic reconnection requires the recruiter to have configured LinkedIn 2FA in **Authenticator-app mode** — a real-world onboarding instruction worth surfacing in the UI regardless of which auth method we use.

5.1 The eleven status events

EVENT	DOCUMENTED MEANING	OUR MAPPING & ACTION
`creation_success`	"The account was successfully added and the initial data synchronization has begun."	→ `CONNECTING`. Persist `account_id`. Blue badge.
`creation_fail`	The connection attempt failed.	→ `CONNECT_FAILED`. Amber, "Try again". **Missing from the current plan.**
`connecting`	"The account is attempting to connect." Temporary — initial connection, provider downtime, or sync catch-up.	→ `CONNECTING`. Note this can recur *after* a healthy period; do not treat it as first-connect-only.
`ok`	"Everything seems to be in good order."	→ `OK`. Green.
`sync_success`	Resynchronisation finished. LinkedIn / IMAP / WhatsApp / Instagram / Telegram only. **LinkedIn Premium sends two payloads**, distinguished by `product`.	→ `OK`. Set `lastSyncedAt`. **Must be idempotent.**
`reconnected`	"Account has been reconnected with success."	→ `OK`. Clear any reconnect nag.
`credentials`	"Synchronization … interrupted due to invalid or missing credentials." Triggers: password change, revoked authorisation, token expiry (LinkedIn/Instagram after one year), session termination, **excessive API requests**, proxy issues, security checkpoints.	→ `RECONNECTION_NEEDED`. Red + email + reconnect link. **Also emit an `Escalation`** — "excessive API requests" means this can be *our* fault.
`permissions`	OAuth permission problem.	→ `PERMISSION_REVOKED`. Red, "Re-grant access". **Missing from the current plan.**
`error`	"Account synchronization has stopped due to an unexpected error during data fetching."	→ `SYNC_STOPPED`.
`stopped`	Synchronisation stopped.	→ `SYNC_STOPPED`.
`deleted`	"You deleted the account."	→ `DISCONNECTED`. Free the seat for future peaks (§9).

5.2 The two payload shapes, side by side

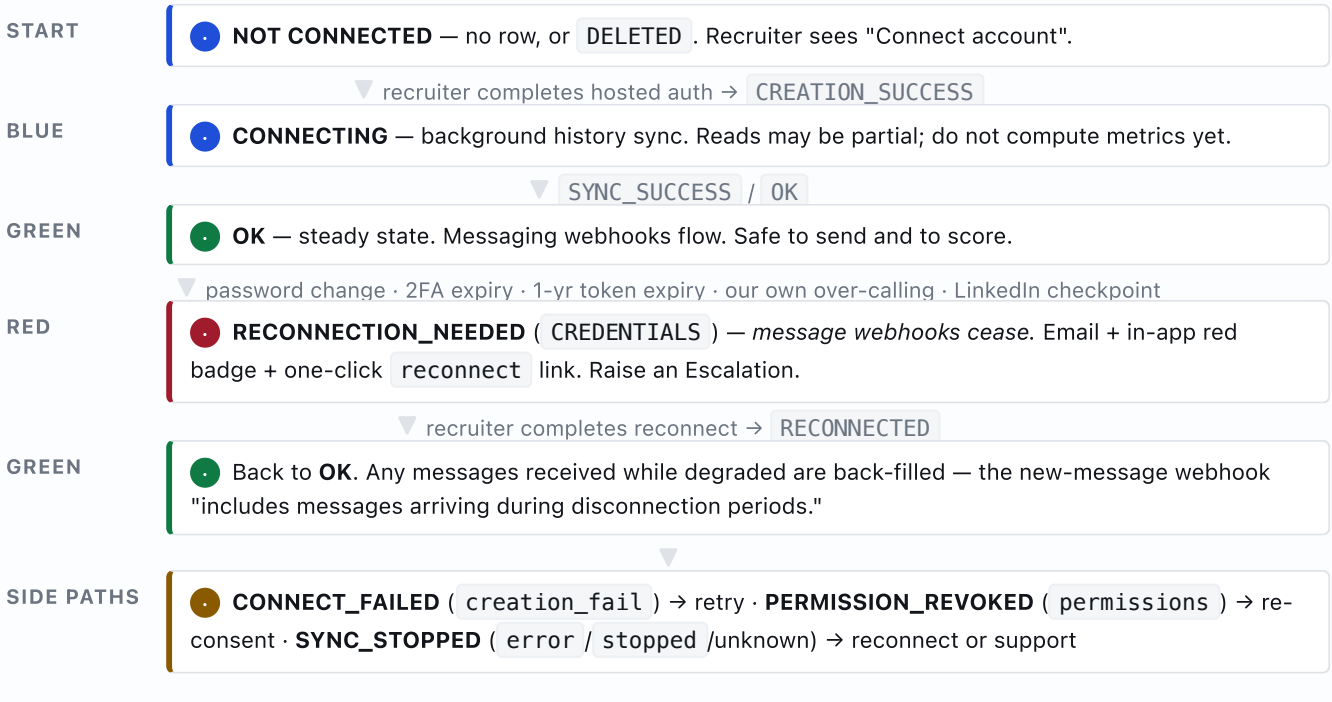
	HOSTED-AUTH `NOTIFY_URL`	REGISTERED `ACCOUNT_STATUS` WEBHOOK
Set up by	The `notify_url` field on the link request. Per-link.	`POST /api/v1/webhooks`, once. Tenant-wide.
Fires for	Only `CREATION_SUCCESS` / `RECONNECTED` for that link.	All eleven events, for the whole account lifetime.
Shape	`{ "status": "CREATION_SUCCESS", "account_id": "e54m8LR22bA7G5qsAc8w", "name": "myuser1234" }`	`{ "AccountStatus": { "account_id": "h_EKCy2lRLef5NzHp0iw4A", "account_type": "LINKEDIN", "message": "CREDENTIALS" } }`
Status key	`status`	`AccountStatus.message`
Carries `name`	**Yes** — the only correlation to our user.	**No.** Correlate by `account_id` against our table.
If you handle only this one	Accounts connect but never heal, never go green from sync, never show as broken.	Accounts heal and report — but you never learn which user a *new* account belongs to.

DESIGN CONCLUSION

Point **both** at one endpoint and branch on shape. `notify_url` is the *only* place we learn the user↔account mapping, so it must write the mapping row; the registered webhook is the *only* place we learn about degradation, so it must own status transitions. Neither is optional. Both must be idempotent.

5.3 State machine

FIGURE 2 — ACCOUNT STATE TRANSITIONS AND WHO TRIGGERS THEM



6.1 Creating a webhook

```
POST https://{YOUR_DSN}/api/v1/webhooks
X-API-KEY: <key>
content-type: application/json

{
  "request_url": "https://app.g3.example/api/unipile/webhook",
  "source": "account_status",
  "name": "g3-account-status",
  "format": "json",
  "enabled": true,
  "account_ids": [],                                  // omit/empty = all accounts
  "headers": [
    { "key": "Content-Type",  "value": "application/json" },  // REQUIRED — see below
    { "key": "X-G3-Webhook-Secret", "value": "<long random secret>" }
  ]
}

// 201 → { "object": "WebhookCreated", "webhook_id": "…" }
```

DOCUMENTED GOTCHA THAT WILL COST AN AFTERNOON

"webhook created by API does not contain header content type JSON by default." A webhook created through the API sends no `Content-Type` unless you add it in `headers`. Most frameworks — including h3/Nitro under TanStack Start — will then refuse to parse the body, and you will see empty payloads with no error. **Always pass the `Content-Type` header explicitly.** Webhooks created in the dashboard UI do not have this problem, which is exactly why it is easy to miss when moving from manual testing to provisioned infrastructure.

6.2 Sources and events

SOURCE	EVENTS	RELEVANCE TO PROJECT BEACON
`account_status`	`creation_success, creation_fail, deleted, reconnected, sync_success, stopped, ok, connecting, error, credentials, permissions`	**Phase 1 — mandatory.** Drives the badge and self-healing.
`messaging`	`message_received, message_read, message_reaction, message_edited, message_deleted, message_delivered`	**Phase 2.** Feeds `ConversationMessage` and inbound `InteractionEvent` rows — i.e. the reply-rate and SLA metrics.
`users`	`new_relation`	**Phase 3.** Accepted LinkedIn invitation → advance `LeadStage` from `CONTACTED`.
`email`	`mail_sent, mail_received, mail_moved`	Phase 4, if we route email through Unipile rather than Resend.
`email_tracking`	`mail_opened, mail_link_clicked`	Feeds `RecruiterKpiSummary.emailOpenRate` — which the schema already correctly annotates with the Apple Mail Privacy Protection caveat. Directional only.
`calendar_event`	`calendar_event_created, _updated, _deleted`	Could replace manual `ManualActivityLog` interview entry. Nice-to-have.

Two useful extras on the create call: `account_ids` scopes a webhook to specific accounts (handy for staging vs production on one tenant), and `data` lets you rename payload properties (`{ "name": "<your name>", "key": "<original>" }`) — a feature to *avoid*, because it makes payloads diverge from the docs for whoever debugs it next.

6.3 Delivery semantics

Success criterion	HTTP **200** returned in **under 30 seconds**.
Retries	**5 attempts** on any non-200, with incremental / exponential delay.
Consequence	At-least-once delivery. Duplicates are normal — reinforced by LinkedIn Premium's double `SYNC_SUCCESS`.
Observability	Marketing site: delivery attempts with full request/response logs visible in the dashboard, retained **30 days**. MEDIUM
Latency claim	"Delivered in < 500ms" MEDIUM — a marketing figure, not an SLA. Note the contrast with `new_relation`, which is documented as lagging **up to 8 hours**.

Architectural implication. The 30-second budget means the handler must not do real work inline. Validate, persist the raw event, return 200, process asynchronously. On Cloudflare that is `ctx.waitUntil()` for light work or a Queue for anything heavier (§12).

6.4 The signature contradiction

SOURCE	CLAIM	CONF.
OpenAPI reference for `POST /webhooks`	The request schema contains `request_url, source, name, format, enabled, account_ids, headers, data`. **No secret. No signing key. No signature algorithm. No verification field of any kind.**	HIGH
Webhooks overview doc	Security is described purely as user-supplied custom headers, e.g. `{"key":"Unipile-Auth","value":"yoursecretkey"}`. HMAC is not mentioned.	HIGH
`unipile.com/developer-real-time/` (marketing)	"Each webhook request includes a signature header that you can verify using your webhook secret." · "Signature Verification — Every webhook is signed."	MEDIUM

RESOLUTION — DESIGN FOR THE DOCUMENTED CAPABILITY, NOT THE ADVERTISED ONE

The two cannot both be describing the same API version. Possibilities: the marketing page describes a newer/beta capability the reference has not caught up with; it describes a different product tier; or it is aspirational copy. **Do not build verification against a signature scheme whose algorithm, header name and secret provisioning are nowhere documented.**

Ship this instead (all four, defence in depth): **(1)** a high-entropy shared secret in a custom header, set at webhook-creation time and compared with a timing-safe equality check; **(2)** an unguessable path segment — `/api/unipile/webhook/<32-byte random>`; **(3)** treat every payload as untrusted input — the only thing we act on is an `account_id` that must already exist in our table, and a status from a closed allow-list, so a forged payload can at worst re-assert a state for an account we already know about; **(4)** raise Appendix-B question 1 with support and add real signature verification the moment it is documented.

7 · WHAT YOU CAN ACTUALLY DO ONCE CONNECTED

All HIGH. Mapped to the Project Beacon models each one feeds.

7.1 Messaging

OPERATION	CALL	NOTES / BEACON MAPPING
Start a 1-to-1 chat (first touch)	`POST /api/v1/chats` `account_id, attendees_ids, text`	Creates the chat if absent and syncs attendees. `attendees_ids` is the recipient's **provider internal id** — you must resolve it first (§7.3). → `Conversation` + outbound `InteractionEvent`.
Reply in an existing chat	`POST /api/v1/chats/{chat_id}/messages` `multipart: text, attachments`	→ `ConversationMessage(sender: ME)`.
Group chat	`POST /api/v1/chats`	Multiple `attendees_ids` + optional `subject`.
LinkedIn InMail	+ `linkedin[api]=classic` + `linkedin[inmail]=true`	Premium only. Check balance first via `GET /linkedin/inmail_balance`. Reaches people you are not connected to — the workhorse for cold recruiting.
Attachments	`attachments=@file`	~15 MB max; PDF/image/video. Inbound attachments come as `att://` URLs to be resolved via the attachment endpoint.
Read history	`GET /chats · /chats/{id}/messages · /messages`	Cursor-paginated.
Other	`PATCH/DELETE message · reactions · forward · sync chat history`	Full CRUD is available; we need very little of it.

THE ECHO PROBLEM, AND ITS DOCUMENTED FIX

Every message we send comes straight back to us as a `message_received` webhook. Persist it naively and every outbound message is double-counted, inflating `outreach_volume` and corrupting reply-rate. The documented discriminator: *"compare `account_info.user_id` with `sender.attendee_provider_id` to determine if the linked account sent the message or received it."* Equal → we sent it; reconcile against the outbound row we already wrote rather than inserting. This single rule protects the integrity of the whole scoring rubric.

NEW-MESSAGE WEBHOOK PAYLOAD FIELDS

`account_id` · `account_type` · `account_info{type, feature, user_id}` · `event` · `chat_id` · `timestamp` · `webhook_name` · `message_id` · `message` · `sender{attendee_id, name, provider_id, profile_url}` · `attendees[]` · `attachments[{id,size,mimetype,type,url}]` · `reaction` / `reaction_sender` (reaction events only).
History from before connection is excluded; messages that arrived while the account was disconnected *are* included on recovery.

7.2 LinkedIn search — lead sourcing

```
POST /api/v1/linkedin/search?account_id={id}
{ "api": "classic" | "sales_navigator" | "recruiter",
  "category": "people" | "companies" | "jobs" | "posts",
  "keywords": "medical translator", "tenure": [{ "min": 3 }], "profile_language": ["en"] }

// or simply paste a LinkedIn/Sales-Navigator search URL:
{ "url": "https://www.linkedin.com/sales/search/people?query=…" }
```

Facet ids (location, industry, skill…) come from `GET /api/v1/linkedin/search/parameters?account_id=…&type=LOCATION&keywords=…&limit=100`. Results: `{ items[], paging{start,page_count,total_count}, cursor }`. A people item carries `type, id, name, headline, location, profile_url` and — importantly — **`network_distance`** (e.g. `DISTANCE_2`).

Beacon mapping. This is a genuine second sourcing channel alongside the existing ProZ/Bright Data pipeline: `Lead.source = LINKEDIN`, `profileLink`, `linkedinUrn`, and `linkedinMatchConfidence` for the "Danny M" partial-identity case the schema already models. The URL-paste mode is the pragmatic win — a recruiter can hand over a Sales Navigator search they already trust instead of us rebuilding their query in our UI.

7.3 Profiles & enrichment

`GET /api/v1/users/{identifier}?account_id=…&linkedin_sections=*` where `identifier` is a `public_identifier` (`"satyanadella"`) or a `provider_id` (`"ACoAAAEkww…"`). Returns names, headline, location, `network_distance`, `connections_count`, `follower_count`, the `is_premium / is_influencer / is_creator / is_open_profile` flags, education, work experience, recommendations, pictures.

Two operationally critical facts. First, the documented sequence is *search → store public id → fetch profile shortly before sending*, because the profile call is what converts a public identifier into the private `provider_id` that `attendees_ids` needs, *and* refreshes `network_distance` so you choose invitation vs InMail vs plain message correctly. Second, that call is capped at roughly **100 profiles/account/day** — so profile fetching, not messaging, is usually the binding constraint. Second-order note: open SDK issue #18 reports `users.getProfile` mis-serialising `linkedin_sections` when given an array — another reason to call REST directly and pass `linkedin_sections=*`.

7.4 Invitations and acceptance detection

`POST /api/v1/users/invite` to send; list/cancel endpoints exist for sent and received. Acceptance can be detected three ways, and the docs are refreshingly candid about the trade-offs:

METHOD	BEHAVIOUR
`new_relation` webhook	Not real-time — "LinkedIn does not support real-time events." Unipile polls the relation list at random intervals, so **the event may fire up to 8 hours after acceptance.**
Invitation-with-note + message webhook	**Effectively real-time.** "Accepting the invitation generates a new chat containing only your invitation message" — which arrives as a normal `message_received`. *Always send invitations with a note* and you get near-instant acceptance detection for free.
Manual polling	Diff the relations / sent-invitations list. Docs: "space out these checks only a few times by day with random delay, not at fixed time." Fixed-interval polling is itself an automation signal.

Beacon mapping. Acceptance → `StageHistory` row, `LeadStage` `CONTACTED → REPLIED`. Use the note-based path as primary, the webhook as reconciliation.

7.5 Also available (not needed for v1)

Recruiter hiring projects · job postings (create/edit/publish/close, applicants, résumé download) · company profiles · posts, comments and reactions · skill endorsements · contract selection · InMail balance · `GET /linkedin/raw` passthrough to arbitrary LinkedIn endpoints · calendars and events · email send/list/folders/drafts/contacts. The job-posting and applicant-résumé endpoints are worth a second look later — they overlap meaningfully with the recruiting workflow.

THE FRAMING THAT MATTERS

"Unipile does not enforce any limits on your usage — you can send and receive as many messages or events as you need." Every number below is **LinkedIn's** (or Google's, or Meta's). Unipile relays their errors; it does not shield you from them. Product commitments about outreach volume must be derived from this table, not from our queue's capacity.

8.1 LinkedIn HIGH

ACTION	DOCUMENTED LIMIT	CONSEQUENCE / HANDLING
Connection invitations (paid/active account)	**80–100 per day**, and **~200 per week**	The weekly cap binds first. Unipile's own recommendation: **30–50 per day at random intervals on working days**. Over-limit → HTTP **422** `cannot_resend_yet`.
Invitations (free account)	~5 per month *with* a note; 150/week without	Recruiters on free LinkedIn essentially cannot run note-based outreach. Since §7.4 depends on notes for real-time acceptance detection, **a paid LinkedIn seat is a functional prerequisite**, not a preference.
Profile retrieval	**~100 per account per day** standard; Recruiter Lite officially 2,000/day	**Usually the binding constraint.** Cache profiles aggressively; only refresh immediately before a send.
Search results per query	**1,000** classic · **2,500** Sales Navigator / Recruiter	Hard ceiling per query. Broad searches must be sliced by facet, not paged past the cap.
Total profiles surfaced per day	1,000 standard / 2,500 SN-Recruiter recommended	Sourcing budget per account per day.
Profile visits	80–100/day (Premium) · 150/day (Sales Navigator)	Counts toward the same behavioural budget.
InMail	Subscription-dependent (Career = 5/month; Recruiter contracts vary). Free InMail to *open profiles*: **max 800/month/account**	Recommendation: **30–50/day** with random spacing. Check `inmail_balance` before sending.
All other routes	**100/day/account** default	A quiet catch-all that is easy to breach with an over-eager reconciliation job.
Relations / invitation polling	Avoid fixed intervals. After initial sync, fetch "first page only a few times a day with randomly spaced intervals"	Prefer webhooks. Cron-on-the-hour polling is itself a detection signal.
Errors on breach	HTTP **429** or **500**	Back off with jitter. Repeated breach is a documented trigger for a `CREDENTIALS` status — i.e. **over-calling gets the recruiter's account disconnected.**

ACTION	DOCUMENTED LIMIT	CONSEQUENCE / HANDLING
LinkedIn **Recruiter**	Multiple simultaneous sessions trigger disconnection prompts	If a recruiter uses Recruiter in their browser while we hold a session, they get logged out. Mitigate with `disabled_features: ["linkedin_recruiter"]` or cookie auth. **Ask about this before onboarding any Recruiter-seat user.**
New / inactive accounts	"Start low, increase gradually"; random intervals; avoid chaining calls	Implies a per-account **warm-up ramp** in our sender, not a global constant.

THE ARITHMETIC, DONE ONCE, FROM UNIPILE'S OWN OUTREACH GUIDE

1,000 target profiles ÷ 30–50 invitations/day on working days ≈ **5 weeks per account**. With ten connected recruiters that is ~10,000 targets per five weeks — *if* every account is healthy and warmed. This number, not our infrastructure, is what any client-facing capacity promise has to be built on. It also means the outreach queue needs per-account daily/weekly counters with the week boundary tracked explicitly, and a scheduler that jitters send times rather than draining at midnight.

8.2 Other providers HIGH

PROVIDER	OFFICIAL LIMIT	UNIPILE'S RECOMMENDATION
Gmail	500 emails/day	50–100/day. New accounts: 20–50, ramping.
Google Workspace	2,000/day	100–150/day, up to 300. Spread API calls; concurrency is limited.
Outlook	5,000 recipients/day · 500 recipients/message · 1,000 non-relationship recipients/day	Varies with account history and subscription.
WhatsApp	—	Do not use brand-new numbers — new accounts can be "blocked after only 2–3 new chats". Wait **up to 24h after connecting** before any outreach. Never send at intervals shorter than **10–20 seconds**. Limit new-chat creation.
Instagram	—	Max **100 actions/day** and **10/hour**. Random spacing in working hours.
Facebook / Messenger	—	10–50 requests/day; never >50 DMs/day; start at 15/day and add 5 per week. Recipients need >10 mutual connections.
IMAP	Provider-specific	Consult the host's documentation.

Relevant to the existing schema: `ConversationChannel` includes `SMS`, which **Unipile does not provide** — that channel will need a separate vendor (Twilio or similar) or should be dropped. Conversely the enum lacks `EMAIL`, `TELEGRAM` and `MESSENGER`, all of which Unipile does support.

9.1 Pricing MEDIUM (commercial pages, not versioned docs — re-confirm at contract time)

Trial	7 days, no credit card.
Minimum	**€49/month, includes up to 10 linked accounts.**
11–50 accounts	≈ €5.00 per account per month. Tiers continue at 51–200, 201–1K, 1K–5K, 5K+ (custom).
Unit of billing	The **linked account**, not the person. "3 Emails + 2 LinkedIn + 6 WhatsApp = 11 accounts × 5€ = 55€."
Bundling	"Gmail & Outlook = Email + Calendar = 1 billed account."
Usage fees	**None.** No per-request or per-message charge. Attractive for a high-volume outreach product.
Invoicing	Post-paid at the end of each 30-day period, on the **peak number of accounts linked simultaneously** in that period.
Conflicting figure	One third-party source states "$79/mo minimum for 5 accounts" LOW, against €49/$55 for 10 from Unipile's own pricing page and one review. Treat the vendor figure as authoritative.

WHY PEAK BILLING CHANGES THE DESIGN

Under peak billing, a script that connects 40 test accounts for ten minutes sets the month's invoice at 40 accounts. Three consequences, all of them engineering work rather than finance policy: **(1)** seat capping must be enforced *server-side at link-generation*, before Unipile ever sees an attempt; **(2)** use a **separate Unipile tenant/DSN for staging**, or test peaks pollute production invoices; **(3)** record a `peakAccountsThisPeriod` counter of our own so the owner can see what they will be billed *before* the invoice, and so "delete a seat" can honestly say "this frees capacity from next period" rather than implying an immediate refund. The existing plan is one of very few first drafts to spot this — it just needs the counter to make it visible.

9.2 Security & compliance posture MEDIUM

CONTROL	STATED POSITION
Certifications	**SOC 2 Type II** (security, availability, confidentiality), independently audited. **CASA Tier II** (Google Cloud Application Security Assessment).
GDPR	Acts as **Data Processor**; **DPA available on request** — obtain and sign it before production.
Hosting	**France only, on Scaleway.** "No transfers outside EU."
Encryption	At rest: **AES-256-GCM** via Scaleway Key Manager. In transit: TLS. Asymmetric: RSA-OAEP 2048/3072/4096 where required.
Engineering controls	Systematic code review, protected branches, managed secrets, least privilege, mandatory MFA on internal tools, restricted production access.
Pen testing	Annually, by independent third parties.
Not documented	Data retention periods; the sub-processor list; and — notably — **the specific mechanism by which provider credentials and sessions are stored**. Since delegating credential custody is the entire value proposition, this belongs in Appendix B.

Assessment. This is a stronger posture than most tools in this category, and EU-only hosting is a genuine advantage for a platform processing EU candidate PII. Two caveats worth writing down: single-region hosting is also a *single point of failure* with no documented multi-region DR; and SOC 2 Type II plus a DPA does not transfer **LinkedIn** platform risk to Unipile — that risk stays with the recruiter's account and, reputationally, with us (§11).

10.1 The silence is the finding

REDDIT · STACK OVERFLOW · HACKER NEWS — ESSENTIALLY NOTHING

Direct Reddit fetches were blocked in this environment, and every site-scoped and subreddit-targeted search returned zero genuine threads — search engines substituted alternative-listing and comparison sites instead. No Stack Overflow question about Unipile surfaced either. I am reporting that plainly rather than presenting comparison-blog content as community opinion.

Why it matters operationally. For most infrastructure choices, the public corpus is a free debugging resource — you hit an error, someone has already posted it. Here that resource does not exist. Three consequences to plan for: **(1)** vendor support responsiveness becomes a hard dependency — test it deliberately during the 7-day trial by filing a real question (start with Appendix B); **(2)** budget more time for empirical discovery, because undocumented behaviour will be found by us, not looked up; **(3)** keep an internal runbook of every quirk found — it is the only corpus we will have. The counterweight is that Unipile publishes an unusually complete, machine-readable doc set (`llms.txt` + `.md` endpoints + full OpenAPI + an MCP server), which is why §3–§9 could be sourced almost entirely at HIGH confidence. Good docs, thin community.

10.2 GitHub — the official SDK's issue tracker MEDIUM

The most credible practitioner signal available: real developers hitting real edges in `unipile/unipile-node-sdk`. Six open, three closed.

#	ISSUE	WHAT IT TELLS US
18	*Open* — `users.getProfile` serialises `linkedin_sections` incorrectly when an array is provided	Wrapper-layer bug on a route we *will* use for enrichment. Pass `linkedin_sections=*`, or call REST directly.
17	*Open* — `TypeSystemDuplicateTypeKind on EncodedQueryCursor`	Type-level defect around pagination.
16	*Open* — `UnsuccessfulRequestError` does not expose `body.{status,type}`	**The most consequential.** The machine-readable error taxonomy (§3) is hard to branch on through the SDK — exactly what a retry/backoff layer needs.
15	*Open* — `connectLinkedinWithCookie` doesn't expose `ip`	SDK lags the REST API's parameter surface.
14	*Open* — feature request: send/receive as a LinkedIn Page	Company-page messaging is not supported. Relevant if G3 ever wants to message from a brand identity.
13	*Open* — types not exported from the SDK root	DX friction in a TypeScript codebase like this one.
8 · 4 · 3	*Closed* — NestJS/CommonJS interop; `only_unreads` ignored in `getAllChats`; type error in `PostNewChatInput`	Issues do get fixed. Volume is low, which given the thin community is ambiguous — either few users or few problems.

CONCLUSION DRAWN FROM THIS TABLE

Four of six open issues are wrapper defects, not API defects — the REST API is fine; the Node wrapper is thin. **Recommendation: skip the SDK.** Write a ~150-line typed `fetch` client in `server/lib/unipile/client.ts`. It removes a dependency whose known bugs sit precisely on our critical paths (error branching, pagination, profile enrichment), it keeps us aligned with the OpenAPI spec which is the better-maintained artefact, and it works cleanly on Cloudflare Workers where the SDK's Node-18 assumptions are a liability (§12).

10.3 Third-party reviews

SOURCE	SUBSTANCE	CONF.
Salesforge, *Unipile Review 2026*	The most substantive critique found, and useful precisely because it is unflattering. Strengths: "Fast to ship with SDKs, webhooks, and hosted auth that get you sending in days"; messages come from the user's real account; native MCP integration. Gaps: **no lead sourcing, no sequencing or campaign logic, no deliverability infrastructure (warm-up, DNS, placement monitoring), no reporting dashboard, and "user owns account-safety risk."** Verdict quote: *"The API handles sending, but leads, mailboxes, warm-up, sequencing, and reply handling are still on you to build."* And: *"an API is a gift when you are building. It is a delay when you are selling."* Names recruiting tools and AI agents as good fits.	MEDIUM
outx.ai comparison	Places Unipile among the top-5 LinkedIn API alternatives; characterises it as "multi-channel messaging… **limited data access** (messaging-focused, not a full LinkedIn data API)". Correctly identifies the fit as SaaS embedding DMs alongside email/WhatsApp. Also carries the conflicting "$79/mo for 5 accounts" figure.	LOW
yalc.ai, *Unipile vs PhantomBuster vs HeyReach*	Architecturally the clearest of the comparison blogs: Unipile is an API layer you build on; PhantomBuster sells execution time through "a remote Chromium instance with a headless fingerprint that is detectable"; HeyReach runs real sessions on residential setups. Directionally consistent with the primary docs on the key point — a server-side API at human rates presents a different fingerprint from a headless browser.	LOW
G2	Direct fetch returned **HTTP 403**. Two third parties cite different aggregates — 4.3/5 and 4.6/5. Unverified either way.	UNVERIFIED
Anecdote in circulation	"One builder reported getting it running in 15 minutes, then had an agent handling 250 messages," with praise for engineering support. Plausible and consistent with the docs' simplicity, but unattributed and unverifiable.	UNVERIFIED

THE SINGLE MOST USEFUL SENTENCE IN THE ENTIRE SECONDARY CORPUS

"You do not want a profile-search endpoint… You want a connection request sent, then a wait, then a message if they accept." That gap between endpoints and outcomes **is our build**. Unipile supplies the verbs; the sequencing engine, the per-account rate governor, the warm-up ramp, the reply classifier and the recovery flows are Project Beacon's work. Anyone scoping this as "wire up an API" will under-estimate it by an order of magnitude — §12's phase structure is deliberately shaped around that.

11 · RISK REGISTER

RISK	SEV.	EVIDENCE	MITIGATION
R1 LinkedIn restricts a recruiter's account	HIGH	Unipile's own docs list "excessive API requests" and "security checkpoints" as `CREDENTIALS` triggers, and describe 429/500 on limit breach. This is documented vendor behaviour, not speculation. HIGH	Hard per-account governor below documented limits (30–50 invites/day, not 80–100). Randomised spacing. Per-account warm-up ramp. Mixed read/write patterns rather than pure-write bursts. Circuit-breaker: on 429/422, pause that account and alert. Never let a queue drain at full speed.
R2 Both callback shapes not handled	HIGH	§5.2. Two documented shapes, different status keys. HIGH	Correction C1 + C2. Contract test that replays both documented payloads verbatim.
R3 Webhook authenticity cannot be verified as documented	MED	§6.4 — OpenAPI has no signature field; marketing claims every webhook is signed. Direct contradiction. HIGH / MEDIUM	Shared-secret header (timing-safe compare) + unguessable path + treat payloads as untrusted (only act on known `account_id` + allow-listed status). Appendix B Q1. Add real signature verification when documented.
R4 Peak billing surprise	MED	"Billing is based on the peak number of linked accounts active simultaneously during that period." MEDIUM	Server-side seat cap at link generation; separate staging DSN; our own peak counter surfaced to the owner before invoicing.
R5 Vendor platform risk / enforcement against tooling	MED	Multiple 2026 blogs report LinkedIn removing HeyReach's company page and banning its founder's profile in March 2026, and an unattributed "~40% of cloud-proxy automation accounts restricted in Q1 2026." **Every one of these sources is a competitor or affiliate of the tools discussed**, the statistic traces to a single unverifiable analysis, and the ban concerned a *company page*, not the API. LOW	Do not architect on this. But the underlying structural point is sound and cheap to hedge: keep provider-specific logic behind one internal interface (`server/lib/outreach/`) so a vendor swap is a driver change, not a rewrite. Do not couple our data model to Unipile's field names beyond the id columns.
R6 Thin public community	MED	§10.1. HIGH (a verified absence)	Test support during the trial with a real question. Keep an internal quirks runbook. Prefer REST + OpenAPI over the SDK so we debug against the best-maintained artefact.
R7 Single-region dependency	LOW	"France (Scaleway)… No transfers outside EU." A compliance strength and an availability concentration. MEDIUM	Degrade gracefully: queue outbound sends and retry rather than surfacing errors. Never block a recruiter's core workflow on Unipile being up. Ask about DR/SLA (Appendix B Q7).
R8 Silent metric corruption	MED	Two mechanisms, both documented: our own sends return as `message_received` (§7.1), and message webhooks *cease* on non-`OK` status (§5). HIGH	Apply the `account_info.user_id` vs `sender.attendee_provider_id` discriminator. Record account-degradation windows and feed them into `RecruiterScoreSnapshot.excludedMetrics` so SLA metrics are not computed over blind periods. **This one is invisible when it goes wrong** — recruiters get scored on data that silently stopped arriving.

ON LINKEDIN'S TERMS OF SERVICE — STATED PLAINLY

This is unofficial, credential-delegated access to LinkedIn. LinkedIn's User Agreement disfavours automated access and third-party session use, and hosted auth does not change that — it changes *who custodies the credentials*, not whether the access is sanctioned. The honest position: the recruiter is delegating access to their own account, of their own volition, and the residual risk lands on their account. Two obligations follow that are cheap now and expensive later: **(1)** say so explicitly in the connect flow — a short, non-buried consent screen stating that G3 will act on their behalf via a third party and that LinkedIn may restrict accounts that exceed its limits; **(2)** keep volumes conservatively inside documented limits, because the mitigation for platform risk is behavioural, not contractual. Neither of these is legal advice — if G3 is contracting with enterprise clients, this belongs in front of counsel.

12.1 What already exists

EXISTING ARTEFACT	RELEVANCE
`server/Project_Beacon_DB_schema_V2.prisma`	Prisma + PostgreSQL. Already carries `InteractionChannel.LINKEDIN_DM`, `ConversationChannel`, and a `deliveryStatus` comment explicitly naming Unipile webhooks. **No `Organization` model** (see C5). Has `SystemConfig{key,value,notes}` — use it.
`client/` (TanStack Start, React 19)	File-based routing, `@tanstack/react-start ^1.168`. Server routes live alongside page routes. `owner.settings.tsx` / `contractor.settings.tsx` are the natural homes for the connect UI.
`client/vite.config.ts` + `.wrangler/`	Nitro building for a **Cloudflare** target. This drives the two constraints below.
`client/src/lib/auth.tsx`	Auth is currently a **localStorage mock**. Real server-side sessions are a hard prerequisite — a webhook cannot be trusted to identify a user, and the connect endpoint must be authenticated.
`client/src/lib/feature-flags.ts`	Gate the whole integration behind a flag for phased rollout.

TWO CLOUDFLARE CONSTRAINTS TO RESOLVE BEFORE PHASE 1 — BOTH ARE PREREQUISITES, NOT DETAILS

1. Prisma does not run on Workers unmodified. It needs `nodejs_compat` in `wrangler.jsonc` plus either a driver adapter (`@prisma/adapter-pg` / Neon serverless) or Prisma Accelerate, and `prisma generate --no-engine` to keep the client under the Worker size budget. Alternatively put the webhook handler and Prisma on a small Node service and keep only the UI on Cloudflare. **Decide this first** — it shapes where the webhook endpoint lives. MEDIUM

2. Unipile's DSN uses a non-standard port (`apiXXX.unipile.com:13081`), and Workers restrict outbound ports. Unipile documents the workaround: call `https://apiX.unipile.com/api/v1/…?port=XXXXX` over 443. **Verify this end-to-end from a deployed Worker on day one** — it is a five-minute test that de-risks the entire transport, and finding it broken in week three would be painful. HIGH

12.2 Target module layout

```
server/lib/unipile/
  client.ts        # ~150-line typed fetch wrapper: DSN + X-API-KEY, port workaround,
                   # error mapping to the §3 taxonomy, retry w/ jitter. NOT the npm SDK (§10.2).
  status.ts        # extractUnipileStatus() + normalizeUnipileStatus()   <- §2.3
  hosted-auth.ts   # mintConnectLink() / mintReconnectLink()
  webhooks.ts      # provisionWebhooks() — idempotent bootstrap, run at deploy
  handlers.ts      # applyAccountStatusEvent() — the only writer of account state

server/lib/outreach/            # vendor-agnostic boundary (R5)
  governor.ts      # per-account daily/weekly counters, warm-up ramp, jittered scheduling
  driver.ts        # interface: sendMessage / sendInvite / getProfile / search
  driver.unipile.ts

client/src/routes/
  api.unipile.webhook.$token.tsx   # POST — serves BOTH notify_url and account_status webhook
  api.unipile.connect.tsx          # POST — authenticated; mints a link, returns the URL
  api.unipile.disconnect.tsx       # POST — authenticated; DELETE upstream + mark DISCONNECTED
```

12.3 Environment

VARIABLE	PURPOSE
UNIPILE_DSN	`api8.unipile.com:13081`. **Different value per environment** — separate tenant for staging (R4).
UNIPILE_API_KEY	Access token for `X-API-KEY`. Server-only; never exposed to `VITE_*`.
UNIPILE_WEBHOOK_SECRET	Shared secret set as a custom header at webhook creation; compared timing-safely.
UNIPILE_WEBHOOK_PATH_TOKEN	32 random bytes forming the unguessable path segment.
APP_BASE_URL	Public origin used to build `notify_url` and the redirect URLs.

Note the `vite.config.ts` comment: the Lovable config injects `VITE_*` env vars into the client bundle. Unipile secrets must **never** carry a `VITE_` prefix.

12.4 Request/response flow, end to end

FIGURE 3 — CONNECT, THEN STEADY STATE

BROWSER	1 `POST /api/unipile/connect {provider:"LINKEDIN"}` — session cookie authenticated.
SERVER	2 Authorise → **check seat cap** (count `ConnectedAccount` not `DISCONNECTED` vs `SystemConfig['unipile.max_slots']`) → check no in-flight attempt (else 425) → create `UnipileAuthAttempt` nonce (15-min TTL).
SERVER	3 `POST /hosted/accounts/link` with `name=<nonce>`, `single_use`, `bypass_success_screen`, `disabled_options:["cookie_auth"]`, `sync_limit`. Return `{url}`.
BROWSER	4 `window.location.assign(url)` — full page, never an iframe.
UNIPILE	5 `POST notify_url` → **flat** `{status:"CREATION_SUCCESS", account_id, name}`. We resolve nonce → user, upsert `ConnectedAccount` (`CONNECTING`), burn the nonce, return 200 fast.
BROWSER	6 Redirect to `success_redirect_url`. UI polls or subscribes for the badge. **Arrival here is not treated as proof of success** — step 5 is.
UNIPILE	7 Registered webhook → **nested** `{AccountStatus:{account_id, account_type, message:"SYNC_SUCCESS"}}`. Badge → green. **Idempotent** (Premium sends this twice).
STEADY	8 `messaging` webhooks → `ConversationMessage` + inbound `InteractionEvent`. Outbound sends go through `outreach/governor.ts`, never directly.

13 · CORRECTED DATABASE SCHEMA

Additive only — no existing model is mutated. Differences from the previous plan are marked **[C#]** against the corrections in §2.2.

```
// ------------------------------------------------------------------------
// UNIPILE — connected accounts, auth attempts, webhook event log
// ------------------------------------------------------------------------

/// [C6] Mirrors Unipile's hosted-auth `providers` enum exactly.
/// OUTLOOK (not MICROSOFT); MAIL = generic IMAP.
enum AccountProvider {
  LINKEDIN
  GOOGLE
  OUTLOOK
  MAIL
  WHATSAPP
  INSTAGRAM
  TELEGRAM
  MESSENGER
  TWITTER
}

/// [C3] Seven states, not five. CONNECT_FAILED and PERMISSION_REVOKED are
/// distinct because the remedy differs: retry login vs re-grant OAuth consent.
enum AccountStatus {
  CONNECTING           // CREATION_SUCCESS, CONNECTING
  OK                   // OK, SYNC_SUCCESS, RECONNECTED
  RECONNECTION_NEEDED  // CREDENTIALS
  PERMISSION_REVOKED   // PERMISSIONS          [C3]
  CONNECT_FAILED       // CREATION_FAIL        [C3]
  SYNC_STOPPED         // ERROR, STOPPED, and any unmapped string
  DISCONNECTED         // DELETED
}

model ConnectedAccount {
  id               String          @id @default(uuid())
  userId           String          @map("user_id")
  user             User            @relation(fields: [userId], references: [id], onDelete: Cascade)

  // [C5] No orgId. Project_Beacon_DB_schema_V2 has no Organization model;
  // the seat cap lives in SystemConfig['unipile.max_slots'].

  unipileAccountId String          @unique @map("unipile_account_id")
  provider         AccountProvider
  accountEmail     String?         @map("account_email")
  accountName      String?         @map("account_name")

  status           AccountStatus   @default(CONNECTING)
  rawUnipileStatus String?         @map("raw_unipile_status")   // audit trail — keep
  statusChangedAt  DateTime?       @map("status_changed_at")

  // Provider identity — needed to resolve chats/messages and to apply the
  // echo discriminator of §7.1 (account_info.user_id vs sender.attendee_provider_id).
  providerUserId   String?         @map("provider_user_id")
  publicIdentifier String?         @map("public_identifier")
  isPremium        Boolean         @default(false) @map("is_premium")  // drives double SYNC_SUCCESS
[C7]

  lastSyncedAt     DateTime?       @map("last_synced_at")
  lastEventAt      DateTime?       @map("last_event_at")

  // Rate governance (§8). Weekly window tracked explicitly because
  // LinkedIn's ~200/week cap binds before the daily one.
  invitesSentToday    Int       @default(0) @map("invites_sent_today")
  invitesSentThisWeek Int       @default(0) @map("invites_sent_this_week")
  countersResetAt     DateTime? @map("counters_reset_at")
```

```
  weekStartedAt       DateTime? @map("week_started_at")
  warmupStartedAt     DateTime? @map("warmup_started_at")   // per-account ramp
  pausedUntil         DateTime? @map("paused_until")         // circuit breaker on 429/422

  createdAt        DateTime        @default(now()) @map("created_at")
  updatedAt        DateTime        @default(now()) @updatedAt @map("updated_at")

  degradations     AccountDegradation[]

  // [C9] One account per provider per user — prevents accidental seat burn.
  @@unique([userId, provider])
  @@index([status])
  @@index([provider, status])
  @@map("connected_accounts")
}

/// [C9] The nonce passed as hosted-auth `name`. Opaque and single-use:
/// `name` is echoed back through a URL under the recruiter's control, so it
/// must never be the raw user id.
model UnipileAuthAttempt {
  id             String          @id @default(uuid())
  nonce          String          @unique
  userId         String          @map("user_id")
  provider       AccountProvider
  kind           String          // "create" | "reconnect"
  reconnectFor   String?         @map("reconnect_for")   // unipile_account_id
  expiresAt      DateTime        @map("expires_at")      // [C8] now + 15 min
  consumedAt     DateTime?       @map("consumed_at")
  createdAt      DateTime        @default(now()) @map("created_at")

  @@index([userId])
  @@index([expiresAt])
  @@map("unipile_auth_attempts")
}

/// [C7] Idempotency + replay. Webhooks are at-least-once (5 retries), and
/// LinkedIn Premium sends SYNC_SUCCESS twice. Write here first, return 200,
/// then process — this is also the only forensic record we will have (§10.1).
model UnipileWebhookEvent {
  id               String   @id @default(uuid())
  dedupeKey        String   @unique @map("dedupe_key")  // sha256(source|account_id|status|bucketed_ts)
  source           String                               // account_status | messaging | users | ...
  unipileAccountId String?  @map("unipile_account_id")
  payload          Json
  receivedAt       DateTime @default(now()) @map("received_at")
  processedAt      DateTime? @map("processed_at")
  processError     String?  @map("process_error")

  @@index([source, receivedAt])
  @@index([processedAt])
  @@map("unipile_webhook_events")
}

/// [R8] Windows during which an account was not OK. Message webhooks cease
/// while degraded, so any SLA/responsiveness metric computed over an
/// overlapping period is invalid — feed these into
/// RecruiterScoreSnapshot.excludedMetrics rather than scoring a blind window.
model AccountDegradation {
  id                 String           @id @default(uuid())
  connectedAccountId String           @map("connected_account_id")
  connectedAccount   ConnectedAccount @relation(fields: [connectedAccountId], references: [id],
onDelete: Cascade)
```

```
  status             AccountStatus
  startedAt          DateTime         @map("started_at")
  endedAt            DateTime?        @map("ended_at")

  @@index([connectedAccountId, startedAt])
  @@map("account_degradations")
}
```

Changes needed to existing models

MODEL	CHANGE	WHY
`User`	Add `connectedAccounts ConnectedAccount[]`	Back-relation.
`Conversation`	Add `unipileChatId String? @unique` and `connectedAccountId String?`	**Without a unique external id, duplicate webhooks create duplicate conversations.** This is the highest-value single line in the table.
`ConversationMessage`	Add `unipileMessageId String? @unique`	Same reason, per message.
`ConversationChannel`	Add `EMAIL`, `TELEGRAM`, `MESSENGER`. Flag `SMS` — Unipile does not provide it.	Align the enum with what the integration can actually deliver (§8.2).
`InteractionEvent`	Add `unipileMessageId String? @unique`; keep `deliveryStatus` as-is	Ties delivery/read webhooks back to the outbound row and closes the echo loop (§7.1).
`SystemConfig`	Seed `unipile.max_slots`, `unipile.invites_per_day`, `unipile.invites_per_week`, `unipile.warmup_days`	**[C5]** Seat cap and rate governance as data, no migration to retune. Consistent with how the schema already treats `sla_window_hours`.
`Escalation`	No change — `category` is free text	Emit "Unipile Account Disconnected" / "LinkedIn Rate Limit Reached" into the existing owner alert queue.

14 · REFERENCE IMPLEMENTATION

Illustrative, not final. TanStack Start's server-route API shifted across recent versions: current docs use `createFileRoute` with a `server.handlers` object, while `createServerFileRoute` / `createAPIFileRoute` appear in older material. Confirm against the installed `@tanstack/react-start ^1.168.26` before writing the real thing. MEDIUM

14.1 Minting a connect link

```
// server/lib/unipile/hosted-auth.ts
import { unipile } from './client';

const TTL_MS = 15 * 60_000;   // [C8] short — links also die on Unipile's daily restart

export async function mintConnectLink(opts: {
  nonce: string; provider: AccountProvider;
}): Promise<string> {
  const res = await unipile.post('/hosted/accounts/link', {
    type: 'create',
    providers: [opts.provider],                // never "*" — seat/billing control
    api_url: `https://${process.env.UNIPILE_DSN}`,
    expiresOn: new Date(Date.now() + TTL_MS).toISOString(),  // matches the required pattern
    name: opts.nonce,                          // [C9] opaque, single-use
    notify_url:
`${process.env.APP_BASE_URL}/api/unipile/webhook/${process.env.UNIPILE_WEBHOOK_PATH_TOKEN}`,
    success_redirect_url: `${process.env.APP_BASE_URL}/recruiter/settings?connect=ok`,
    failure_redirect_url: `${process.env.APP_BASE_URL}/recruiter/settings?connect=failed`,
    bypass_success_screen: true,
    single_use: true,
    disabled_options: ['cookie_auth', 'proxy', 'autoproxy'],  // no raw li_at pasting
    disabled_features: ['linkedin_organizations_mailboxes'],
    sync_limit: { MESSAGING: { chats: 90, messages: 500 } },  // cuts first-sync time + storage
  });
  return res.url;   // match on `url`, not on res.object (docs spell it two ways)
}
```

14.2 The webhook endpoint — both shapes, one handler

```
// client/src/routes/api.unipile.webhook.$token.tsx
import { createFileRoute } from '@tanstack/react-router';
import { extractUnipileStatus, normalizeUnipileStatus } from '~/server/lib/unipile/status';
import { recordEvent, applyAccountStatusEvent } from '~/server/lib/unipile/handlers';

function timingSafeEqual(a: string, b: string) {
  if (a.length !== b.length) return false;
  let out = 0;
  for (let i = 0; i < a.length; i++) out |= a.charCodeAt(i) ^ b.charCodeAt(i);
  return out === 0;
}

export const Route = createFileRoute('/api/unipile/webhook/$token')({
  server: {
    handlers: {
      POST: async ({ request, params, context }) => {
        // (2) unguessable path  +  (1) shared-secret header    [§6.4]
        if (!timingSafeEqual(params.token, process.env.UNIPILE_WEBHOOK_PATH_TOKEN!)) {
          return new Response('not found', { status: 404 });
        }
        const secret = request.headers.get('x-g3-webhook-secret') ?? '';
        if (!timingSafeEqual(secret, process.env.UNIPILE_WEBHOOK_SECRET!)) {
          return new Response('forbidden', { status: 403 });
        }

        // Content-Type may be ABSENT on API-created webhooks — parse defensively. [§6.1]
        const raw = await request.text();
        let body: unknown;
        try { body = JSON.parse(raw); }
        catch { return new Response('ok', { status: 200 }); }   // never make Unipile retry a malformed
body

        // [C1] BOTH shapes: nested AccountStatus.message OR flat status.
        const rawStatus = extractUnipileStatus(body);
        const accountId = (body as any)?.AccountStatus?.account_id ?? (body as any)?.account_id;
        const name      = (body as any)?.name;            // present ONLY on notify_url

        // [C7] Persist first, dedupe, return 200 fast — 30s budget, 5 retries.
        const { isNew, eventId } = await recordEvent({ body, accountId, rawStatus });
        if (isNew && rawStatus) {
          // Cloudflare: hand off rather than blocking the response.
          context.waitUntil?.(
            applyAccountStatusEvent({
              eventId, accountId, name,
              status: normalizeUnipileStatus(rawStatus),
              rawStatus,
            }),
          );
        }
        return Response.json({ ok: true });   // must be 200 within 30s
      },
    },
  },
});
```

14.3 Bootstrapping the webhook (run once per environment, idempotently)

```
// server/lib/unipile/webhooks.ts   — [C2] without this, nothing after step 5 ever fires
const SOURCES = ['account_status', 'messaging', 'users'] as const;

export async function provisionWebhooks() {
  const existing = await unipile.get('/webhooks');
  const url =
`${process.env.APP_BASE_URL}/api/unipile/webhook/${process.env.UNIPILE_WEBHOOK_PATH_TOKEN}`;

  for (const source of SOURCES) {
    if (existing.items?.some((w: any) => w.source === source && w.request_url === url)) continue;
    await unipile.post('/webhooks', {
      request_url: url,
      source,
      name: `g3-${source}`,
      format: 'json',
      enabled: true,
      headers: [
        // REQUIRED: API-created webhooks send no Content-Type by default. [§6.1]
        { key: 'Content-Type', value: 'application/json' },
        { key: 'X-G3-Webhook-Secret', value: process.env.UNIPILE_WEBHOOK_SECRET! },
      ],
    });
  }
}
```

14.4 Seat cap and the rate governor

```
// server/lib/unipile/seats.ts   — [C9] must run BEFORE minting a link (peak billing, §9.2)
export async function assertSeatAvailable(db, userId: string, provider: AccountProvider) {
  const max = Number(await getSystemConfig(db, 'unipile.max_slots', '10'));
  const used = await db.connectedAccount.count({ where: { status: { not: 'DISCONNECTED' } } });
  if (used >= max) throw new SeatCapReached(used, max);        // notify the owner

  const mine = await db.connectedAccount.findUnique({ where: { userId_provider: { userId, provider } }
});
  if (mine && mine.status !== 'DISCONNECTED') throw new AlreadyConnected(mine.status);
}

// server/lib/outreach/governor.ts  — §8. Nothing sends without passing through here.
export async function canSendInvite(db, acct: ConnectedAccount): Promise<Decision> {
  if (acct.status !== 'OK')                       return deny('account_not_ok');
  if (acct.pausedUntil && acct.pausedUntil > now) return deny('circuit_open');

  const dailyCap = warmupCap(acct.warmupStartedAt);  // e.g. 10 -> 20 -> 35 -> 50 over 4 weeks
  if (acct.invitesSentToday    >= dailyCap) return deny('daily_cap');
  if (acct.invitesSentThisWeek >= 180)      return deny('weekly_cap');  // 200 documented; stay under

  // Randomised spacing — fixed intervals are themselves a detection signal (§8.1).
  return allow({ delayMs: jitter(4 * 60_000, 22 * 60_000) });
}
```

WHY THE GOVERNOR IS A SEPARATE MODULE AND NOT AN IF-STATEMENT

It is the component that decides whether recruiters keep their LinkedIn accounts. It needs its own tests (cap arithmetic, week rollover across timezones, warm-up curve, breaker reset), its own metrics, and the ability to be tuned from `SystemConfig` without a deploy. Every send path — outreach sequences, one-off DMs from the conversations UI, follow-ups from the email queue — must route through it, or the caps are advisory.

15 · PHASED DELIVERY PLAN

Phase 0 — De-risk (before any feature work)

TASK	EXIT CRITERION
Start the 7-day trial; get DSN + access token; provision a **second tenant for staging**	Two DSNs in two env files. R4 contained.
Verify the port workaround from a deployed Cloudflare Worker	A successful `GET /accounts` over 443 with `?port=`. Blocks everything if it fails.
Decide the Prisma-on-Workers strategy (driver adapter vs Accelerate vs separate Node service)	Written decision + a Worker that reads one row.
Replace the localStorage auth mock with server-side sessions	`/api/unipile/connect` can identify a caller. Hard prerequisite.
File Appendix-B Q1 (webhook signatures) and Q4 with support	A reply — which also measures support quality (R6).
Request and sign the DPA	Executed before any real candidate data flows.

Phase 1 — Authentication pipeline (the scope of the document this one supersedes)

- Migration from §13 (`ConnectedAccount` , `UnipileAuthAttempt` , `UnipileWebhookEvent` , `AccountDegradation` , `SystemConfig` seeds).
- `server/lib/unipile/{client,status,hosted-auth,webhooks,handlers}.ts` .
- Three routes: connect / disconnect / webhook. `provisionWebhooks()` wired into deploy.
- Settings UI: seven badge states (§2.4), Connect / Reconnect / Re-grant / Try-again / Disconnect.
- Owner view: seats used vs cap, **peak-this-period**, per-account status.
- Escalations on `CREDENTIALS` , `PERMISSIONS` , seat-cap-reached. Email on red states.
- Whole feature behind a flag in `feature-flags.ts` .

Acceptance: ① connect a real LinkedIn account end-to-end and see blue→green; ② replay both documented payloads verbatim in a contract test and assert correct state; ③ deliver the same event five times and assert exactly one state change; ④ change the LinkedIn password out-of-band, observe `CREDENTIALS` , reconnect in one click; ⑤ attempt to exceed the seat cap and be refused *before* any Unipile call; ⑥ confirm all eleven documented statuses map to a state with no `default` branch taken.

Phase 2 — Inbound messaging

- `messaging` webhook → `Conversation` / `ConversationMessage` , deduped on `unipileChatId` / `unipileMessageId` .
- **Echo discriminator** (`account_info.user_id` vs `sender.attendee_provider_id`) — with a test proving outbound messages are not double-counted.
- Inbound `InteractionEvent(direction: INBOUND, channel: LINKEDIN_DM)` feeding reply-rate and SLA metrics.
- `AccountDegradation` windows exported into `excludedMetrics` (R8).
- Attachment resolution via `att://` .

Acceptance: a real inbound LinkedIn reply appears in the conversations UI within seconds; sending from the UI produces exactly one `InteractionEvent` and one `ConversationMessage` ; a degraded window is visibly excluded from SLA scoring.

Phase 3 — Governed outbound

- `outreach/governor.ts` + `driver.unipile.ts`; nothing sends outside them.
- Send from the conversations UI and from `EmailQueueItem` approvals.
- Invitations **always with a note** (real-time acceptance detection, §7.4); `new_relation` as reconciliation.
- Acceptance → `StageHistory` + `LeadStage` advance.
- Circuit breaker on 429/422; per-account warm-up ramp; jittered scheduling.

Acceptance: the governor refuses the 51st invite in a day and the 181st in a week; a simulated 422 opens the breaker and raises an Escalation; no two sends on one account are less than four minutes apart; a fresh account starts at the warm-up floor, not the ceiling.

Phase 4 — Sourcing & enrichment

- LinkedIn search (URL-paste first — lowest effort, highest recruiter trust) → `Lead(source: LINKEDIN)`.
- Profile enrichment respecting the ~100/day ceiling; cache hard; refresh only just before a send.
- `network_distance` drives invitation vs InMail vs plain message.
- Feed `linkedinUrn`, `linkedinMatchConfidence`, `yoeConfidence`; route ambiguous identities to `FLAGGED_REVIEW` — the "Danny M" path the schema already models.

Phase 5 — Optional

Email via Unipile (evaluate against the incumbent Resend — Unipile gives one inbox model and open/click tracking, but the Salesforge critique on deliverability infrastructure stands and is the reason to be cautious) · calendar events replacing manual interview logging · WhatsApp (mind the 24-hour post-connect wait and 10–20s spacing) · job postings and applicant/résumé endpoints.

Cross-cutting definition of done

- No Unipile field name appears outside `server/lib/unipile/` except the id columns (R5).
- Every webhook write is idempotent and every raw payload is retained (§13).
- Every send passes the governor. No exceptions, no direct driver calls.
- Every unmapped vendor string logs an error and raises an alert — never silently degrades.
- Secrets are server-only; no `VITE_` prefix on anything Unipile.
- Staging and production use different Unipile tenants.

APPENDIX A · SOURCES, WITH CONFIDENCE SCORES

32 sources. Every URL below was fetched or searched during this research pass. Score per §0.

A.1 PRIMARY — UNIPILE DEVELOPER DOCUMENTATION & OPENAPI HIGH · 90–99

SOURCE	SCORE	WHAT IT ESTABLISHED
developer.unipile.com/llms.txt	99	Machine-readable index of the entire doc set — guides, ~110 API reference pages, changelog. Used to drive this whole research pass. Its existence is itself a quality signal.
docs/hosted-auth.md	98	Hosted-auth endpoint, parameter table, `notify_url` flat payload, the three implementation rules (server-side, no iframe, watch CREDENTIALS).
reference/hostedcontroller_requestlink.md	99	Full OpenAPI schema — `anyOf` create/reconnect variants, provider enum, `expiresOn` regex, `single_use`, `disabled_options`, `sync_limit`, daily-restart caveat.
docs/account-lifecycle.md	98	**The nested `AccountStatus` payload (finding C1)**, every status definition, CREDENTIALS triggers, Premium double SYNC_SUCCESS, the "GETs still work but webhooks cease" behaviour.
reference/webhookscontroller_createwebhook.md	99	All six sources, all eleven `account_status` events, full create schema — and the **documented absence of any HMAC/secret field (finding R3)**.
docs/webhooks-2	97	Retry semantics (5 attempts, 200-in-30s), custom-header auth pattern, and the missing-Content-Type gotcha.
docs/provider-limits-and-restrictions.md	97	The whole of §8 — every numeric limit for LinkedIn, WhatsApp, Instagram, Facebook, Gmail, Outlook, plus the Recruiter concurrent-session warning.
reference/accountscontroller_createaccount.md	99	Native-auth bodies for all ten providers; the complete HTTP error taxonomy incl. 425 `auth_in_progress`.
docs/new-messages-webhook.md	97	Message payload fields; **the echo discriminator (§7.1)**; history-exclusion and disconnection-backfill behaviour.
docs/detecting-accepted-invitations.md	97	Three detection strategies; the **up-to-8-hour `new_relation` lag**; the invitation-with-note real-time trick; anti-fixed-interval polling guidance.
docs/how-to-create-outreach-sequence.md	96	Vendor-recommended sequence architecture and the **5-weeks-per-1,000-profiles** arithmetic.
docs/linkedin-search.md	96	Search endpoint, classic/sales_navigator/recruiter modes, URL-paste mode, parameters lookup, result shapes with `network_distance`.
docs/retrieving-users.md	95	Profile endpoint, identifier types, `linkedin_sections=*`, returned fields.
docs/send-messages.md	96	Start-chat vs send-in-chat, `attendees_ids`, the InMail parameter pair, attachment handling.

SOURCE	SCORE	WHAT IT ESTABLISHED
docs/api-usage.md	95	`X-API-KEY`, DSN base URL, cursor pagination, and **the `?port=` workaround (critical for Cloudflare)**.
docs/connect-accounts	95	Hosted vs custom auth; the per-provider auth-feature matrix.
docs/linkedin.md	93	Checkpoint types (2FA/OTP/IN_APP_VALIDATION/CAPTCHA/PHONE_REGISTER), 5-minute intent expiry, `TRY_ANOTHER_WAY`, the Authenticator-app-mode requirement for auto-reconnect.
docs/getting-started	93	Signup, DSN, access-token provisioning.
docs/mcp.md	92	MCP server endpoint and auth — useful build-time accelerant.
docs/native-auth.md	90	Confirms native auth exists and is per-provider; the page itself is thin and defers to provider guides (noted honestly rather than over-read).

A.2 SECONDARY — VENDOR COMMERCIAL PAGES & OFFICIAL SDK MEDIUM · 60–89

SOURCE	SCORE	WHAT IT ESTABLISHED / CAVEAT
github.com/unipile/unipile-node-sdk/issues	88	Nine issues — the most credible practitioner signal available. Drove the "use REST, not the SDK" recommendation. Caveat: an issue tracker over-represents problems.
github.com/unipile/unipile-node-sdk	85	SDK surface, client init, `UnsuccessfulRequestError`, `extra_params` and `client.request.send()` escape hatches.
unipile.com/pricing-api	78	**Peak-based 30-day billing**, €49/10 accounts, €5/account, the bundling rule, 7-day trial. Vendor-authoritative but unversioned marketing — re-confirm at contract time.
unipile.com/security-compliance	76	SOC 2 Type II, CASA Tier II, GDPR processor + DPA, France/Scaleway only, AES-256-GCM, annual pen tests. Self-attested; request the actual report.
unipile.com/developer-real-time	62	Source of the **signature-verification claim that contradicts the OpenAPI spec (R3)**, plus <500ms latency, 5 retries with exponential backoff, 30-day log retention. Scored low *because* of the contradiction.
salesforge.ai/blog/unipile-review	68	The most substantive critique found (§10.3). Specific and checkable, but published by an adjacent commercial vendor.
unipile.com · /developer-auth-on-behalf · /linkedin-api · /how-linkedin-api-pricing-works	60–70	Corroborating marketing pages on positioning, per-account pricing and delegated auth. Used only where they agree with primary docs.
prisma.io/docs/guides/cloudflare-workers · Cloudflare Hyperdrive + Prisma	85	The Prisma-on-Workers constraint in §12.1: `nodejs_compat`, driver adapters, `--no-engine`, Accelerate.

SOURCE	SCORE	WHAT IT ESTABLISHED / CAVEAT
tanstack.com — Server Routes	80	Server-route patterns for the webhook endpoint. Flagged as version-sensitive in §14.

A.3 TERTIARY — COMPARISON & COMMENTARY LOW · 25–59 (DIRECTIONAL ONLY; NOT BUILD INPUTS)

SOURCE	SCORE	USE AND CAVEAT
outx.ai/blog/linkedin-api-alternatives-2026	45	Positioning; "limited data access" characterisation. Competitor content; carries a pricing figure that conflicts with the vendor's own page.
yalc.ai — Unipile vs PhantomBuster vs HeyReach	42	Clearest architectural framing among the comparison blogs. Commercially interested.
connectsafely.ai · yalc.ai · linkedinsider.blog · wonda.sh · joinvalley.co · marketingexpertshub.com	25–40	The 2026 "enforcement crackdown" narrative and the HeyReach incident (R5). Six sources, all with commercial interest in the conclusion, all tracing the headline statistic to one unverifiable analysis. **Recorded, deliberately not load-bearing.**
linkedapi.io/vs/unipile · social-api.ai · zernio.com · thatmarketingbuddy.com · Product Hunt · SaaSHub · AlternativeTo	25–35	Direct-competitor landing pages and listing aggregators. Reviewed for completeness; used for nothing.

A.4 ATTEMPTED AND UNAVAILABLE UNVERIFIED · <25

SOURCE	OUTCOME
g2.com/products/unipile/reviews	**HTTP 403.** Third parties cite 4.3/5 and 4.6/5 — mutually inconsistent, neither verified.
reddit.com (direct fetch + ~4 site-scoped searches)	**Fetch blocked; searches returned zero genuine threads.** Reported as finding §10.1 rather than filled in from elsewhere.
Stack Overflow (targeted search)	**No Unipile questions surfaced.** Same treatment.
"15 minutes to running, agent handling 250 messages"	Unattributed anecdote in circulation. Plausible; unverifiable. Excluded from all reasoning.
"~40% of cloud-proxy automation accounts restricted in Q1 2026"	Attributed to "Northlight.ai" by multiple commercially-interested blogs; primary analysis not locatable. Excluded.

Score distribution. 20 sources at High (all primary Unipile documentation or OpenAPI), 9 at Medium, 13 at Low, 5 Unverified. Everything load-bearing in §3–§9 and §13–§14 rests on the High tier. The Low tier appears only in §10–§11 and is labelled as directional in situ. Where sources conflict — webhook signatures, pricing minimums, G2 ratings — both readings are shown rather than one being chosen silently.

APPENDIX B · QUESTIONS FOR UNIPILE SUPPORT

File these during the 7-day trial. Two purposes: they close real design gaps, and the quality of the answers is the best available measure of the support dependency that R6 identifies.

#	AREA	QUESTION	BLOCKS
1	Webhook security	Your marketing site states "Each webhook request includes a signature header that you can verify using your webhook secret," but the OpenAPI schema for `POST /webhooks` has no secret or signature field, and the webhooks guide documents only user-supplied custom headers. Which is current? If signing exists: what is the header name, the algorithm, the signed payload construction, and where is the secret provisioned?	**Phase 0** — determines whether we ship real verification or the shared-secret fallback.
2	Credential custody	Your security page covers encryption at rest and in transit but not how provider credentials and live sessions are specifically stored and isolated per tenant. Since delegated credential custody is the core value proposition, can you describe it, or share the relevant SOC 2 control descriptions?	Security review / DPA.
3	Billing	Confirm precisely how the 30-day peak is computed: is it the maximum count of non-deleted accounts at any instant, or a daily sample? Does an account in `CREDENTIALS` or `SYNC_STOPPED` count toward the peak? Does deletion and same-period re-connection count once or twice?	Seat-cap design + owner-facing cost forecast.
4	Transport	Confirm the `?port=` over-443 workaround is fully supported and stable for all endpoints including webhook registration — we are deploying to Cloudflare Workers, which restricts outbound ports.	**Phase 0** — hard blocker.
5	Status semantics	Is `creation_fail` ever delivered to the hosted-auth `notify_url`, or only to a registered `account_status` webhook? And does the `notify_url` payload ever use the nested `AccountStatus` envelope?	Handler branching (C1).
6	Idempotency	Is there a stable per-delivery event id we should dedupe on? For LinkedIn Premium's two `SYNC_SUCCESS` payloads, what are the possible `product` values? Are events ordered per account?	Dedupe key design (C7).
7	Availability	Is there a published SLA or status page? Given France/Scaleway-only hosting, what is the DR posture, and what should we expect during a regional incident?	Degradation design (R7).
8	LinkedIn Recruiter	Your docs note that multiple simultaneous sessions trigger Recruiter disconnection prompts. For a recruiter who uses LinkedIn Recruiter daily in their browser, what is the recommended configuration so we do not log them out?	Onboarding for Recruiter-seat users.
9	Rate limits	Do you expose per-account consumption counters, or relay LinkedIn rate-limit headers, so our governor can read remaining quota rather than infer it from local counters? (One source suggests headers are relayed — we could not confirm it in the docs.)	Governor accuracy (§14.4).

Closing assessment

Recommendation: proceed. Unipile is the right shape for Project Beacon — the schema already assumes it, the documentation is unusually complete and machine-readable, the security and GDPR posture is better than most of the category, and the pricing model (per-account, no usage fees) suits a high-volume outreach product. Hosted auth genuinely keeps provider credentials out of our scope, which is the single most valuable thing it does.

Three things should govern how the work is planned. **First**, the two-payload-shape finding (C1) and the missing webhook registration (C2) are the difference between an integration that appears to work and one that actually heals; they are cheap to fix now and expensive to discover in production. **Second**, LinkedIn's per-account ceilings

— not our infrastructure — are the product's real throughput limit, and the rate governor that respects them is a first-class component, not a guard clause. **Third**, the honest reading of the secondary corpus is that Unipile supplies verbs and we supply the system: sequencing, governance, warm-up, reply handling and recovery are all Project Beacon's build. Scope Phase 1 as authentication only, and treat everything after it as the real work.

Prepared 31 July 2026. All primary-source claims verified against `developer.unipile.com` during this research pass; vendor commercial terms and the TanStack/Prisma platform notes should be re-confirmed at implementation time.